技术扫盲 • 约束解码 与 单一事实源

这份文档只讲简历里那两行最容易说不清的：**约束解码、一份 Schema 四处消费**。 目标是让你不仅能背，而是**能画出机制、能答追问**。

## 一、约束解码（Constrained Decoding）

### 1.1 先看清楚问题：模型是怎么”写字”的

大模型不是一次性吐出整段 JSON，而是**自回归**地一个 token 一个 token 往外蹦： 每步算一遍词表上所有 token 的概率，挑一个追加到末尾，再算下一个。

关键在于：**默认情况下词表里任何 token 都可能被选中**。所以自由生成会出这些错：

| 错误 | 表现 |
|---|---|
| 结构断裂 | {"module": "meetin 之后直接结束，引号没闭合 |
| 枚举越界 | "module": "开会" —— 词表里有，但契约里只允许 meeting |
| 多余解释 | JSON 前后带一句”好的，我帮你安排：…” |

**根因：格式约束只存在于训练分布里，不存在于生成机制里。** 模型”倾向于”输出合法 JSON， 但没有任何机制**保证**它。而工具调用是 0 容忍的 —— 解析失败就是功能失败。

### 1.2 约束解码做的事：把非法 token 的概率变成 0

不是生成完再修（那是后处理），而是**在采样那一步就干预**：

每生成一个 token 时：

① 用状态机判断"当前位置允许哪些 token"
② 把不在允许集合里的 token 的 logit 置为 -∞
③ 对剩下的做 softmax，再采样

softmax 里 -∞ 的指数是 0，所以**非法 token 的概率不是”很低”，而是恰好为 0** —— “输出结构非法”在数学上不可能发生。状态机本质上是一个**下推自动机**， 在跟踪”语法走到哪了”：读到 {"module": " 时只允许枚举里的值，读到 duration_min": 后只允许数字。

> **面试可以这么说**：约束解码是在**解码器的采样层**做词表剪枝， 用文法状态机把非法 token 的 logit 屏蔽掉，使输出在语法上不可能违规。

### 1.3 GBNF 是什么

GBNF = GGML BNF，是 llama.cpp 用的**文法描述语言**，写法接近正则 + BNF：

```
root   ::= "{" "\"module\":" module "," "\"title\":" string "}"
module ::= "\"class\"" | "\"exam\"" | "\"meeting\"" | "\"alarm\""
string ::= "\"" [^"]* "\""
```

项目里不用手写 GBNF —— llama.cpp 支持**从 JSON Schema 自动编译**：请求里带 response_format: {type: "json_schema", json_schema: {...}}，服务端先转成 GBNF 再按它约束采样。

**所以我写的是一份 JSON Schema，拿到的是一整套语法约束，不用手维护两套东西。**

### 1.4 它的三个代价（面试爱追问这里）

| 代价 | 说明 |
|---|---|
| **首 token 延迟** | 语法要预编译，Schema 越复杂编译越久 |
| **表达受限** | Schema 没定义的字段，模型**根本生成不出来**。所以契约设计本身就是产品设计 |
| **token 边界问题** | tokenizer 的词可能与语法边界不对齐，状态机要处理”半个 token” |

### 1.5 但格式合法 ≠ 语义正确

约束解码能保证 module 落在枚举里，但**保证不了**”开会”该归 meeting 还是 class。 所以执行层再留两个出口：**clarify**（信息不足就反问）与 **reject**（越界就拒答， 而不是硬凑一个形式合法的答案）。

一句话：约束解码解决”**能不能解析**”，clarify/reject 解决”**该不该执行**”。

## 二、“模型不算日期”：这其实不是格式问题

很多人把这条和约束解码混在一起，其实它解决的是**另一个维度**的问题。

模型做日期算术会在跨月、跨年、闰年、周偏移叠加（“下下周三”）处翻车。 **更致命的是错误是静默的**——错一天还是错一周，界面上都是一个正常日程卡片， 用户不会报错，只会在某天发现事情没做。

所以架构上做了取舍：**契约只允许相对表达，绝对日期由确定性代码算**。

模型输出： {"date_kind": "weekday", "date_value": "WE", "week_offset": 1}
↓ （模型的工作到此为止：只做语义归一化）
确定性代码： 解析成 2026-11-11 09:00
↓
用户看到： 下周三 09:00 开会

**为什么这样更好，而不只是”更保险”：**

| | 让模型算 | 确定性代码算 |
|---|---|---|
| 可测 | 每次输出可能不同，无法穷举 | 可写单元测试，覆盖率可量化 |
| 可复现 | 换模型/换温度结果就变 | 输入确定则输出确定 |
| 出错形态 | 静默、随机、难排查 | 抛异常、有堆栈、能定位 |

**面试可以这么说**：我没有去”努力让模型把日期算对”，而是**把这类能力从模型里移除**。架构的责任划分比模型能力的提升更可靠。

## 三、一份 Schema 四处消费（单一事实源）

### 3.1 先讲问题：契约漂移

同一份数据结构，在项目里有四个消费方：

| 消费方 | 用途 |
|---|---|
| 约束解码 | 生成 GBNF 文法，约束模型输出 |
| LoRA 训练 | 决定训练标签的字段名与枚举值 |
| 执行层校验 | Pydantic 模型校验入参合法性 |
| MCP 工具定义 | inputSchema，告诉客户端这个工具要什么参数 |

如果四处各写一份，**改了一个字段忘了改另一处**，就会发生：

训练时模型学的是 remind_before，推理时校验层要的是 remind_before_min —— 模型输出完全正确，但校验全挂。而且**不报语法错**，是最难查的一类 bug。

这叫**契约漂移（contract drift）**。它不是”某天突然坏”，而是”慢慢不一致直到某次上线炸掉”。

### 3.2 解法：只有一个地方是人写的

schema/schedule_command.v2.json 是**唯一事实源**，其余三处全部从它派生：

| 消费方 | 怎么派生 |
|---|---|
| 约束解码 | 直接把这份 Schema 交给 llama.cpp 编译成 GBNF |
| 训练数据 | 生成器读 Schema 的字段与枚举来构造标签 |
| 执行层 | 写一个**镜像类**（Pydantic），字段一一对应 |
| MCP 工具 | 用 Literal[*SCHEMA_ENUM] **直接取枚举值**， |

| 消费方 | 怎么派生 |
| --- | --- |
| | 不再手抄 |

Pydantic 那处为什么”镜像”而不直接读 Schema？因为运行时要的是类型化对象和逐字段校验，Schema 本身只是 JSON 描述。镜像类无法自动生成，所以**用 CI 断言锁死**。

### 3.3 关键：用 CI 断言把”一致”变成机器保证

- 断言 Pydantic 镜像的字段集合 **等于** Schema 的字段集合
- 断言 MCP 工具的枚举 **等于** Schema 的枚举

这样”训练时一个字段、调用时另一个字段”这类静默失效，**在提交前就被拦截**。

> **面试可以这么说**：单一事实源不是”约定大家改代码时记得同步”， 而是**把一致性写成了 CI 断言**——机器能验的，就不靠人记。

### 3.4 MCP 在这里到底是什么（最容易被问混的点）

很多人把下面三个混为一谈，先把边界划清：

| 概念 | 管什么 | 不管什么 |
| --- | --- | --- |
| **Function Calling** | 模型厂商 API 里”让模型调用函数”的能力 | 不管工具怎么被发现 |
| **提示词工程** | 怎么把话说清楚让模型照做 | 完全不管格式保证 |
| **MCP** | **工具如何被描述、被发现、被调用**的协议 | 不管模型怎么把中文变 JSON |

一句话：**MCP 管”工具层的标准化”，不是”翻译层”。**

它带来的是：任何支持 MCP 的客户端（Claude Desktop、Cursor 等）**零适配接入** —— 客户端通过协议自己 tools/list 拿到工具签名，不用读文档、不用写胶水代码。

而我们的亮点在于：**这个 inputSchema 不是手写的，是跟解码文法、训练标签同源派生的**。 所以”模型能生成的 JSON”与”客户端看到的工具签名”永远一致。

## 四、两块怎么串成一句话

| 技术 | 解决什么 | 一句话 |
| --- | --- | --- |
| 约束解码 | 输出**格式**合法 | 采样层屏蔽非法 token，结构违规概率为 0 |
| 模型不算日期 | 语义**可测** | 把不可测的能力从模型里移除，交给有单测的确定性代码 |
| 单一事实源 | 多处**一致** | 一份 Schema 派生四处消费，一致性由 CI 断言保证 |

> 收尾句：**“我不是在赌模型变强，而是在设计上让模型的错误无处发生、或发生了能被立刻发现。”**

## 五、最可能的三个追问

**Q：约束解码会不会降低模型能力？** A：会限制语法层面（Schema 没定义的生成不出来），但语义空间没变。代价主要在首 token 延迟， 总时长往往反而更短 —— 模型不会在格式上走弯路。

**Q：单一事实源是不是就是”少写重复代码”？** A：不是。省代码是次要的，核心是**消除静默失效** —— 重复代码报错明显， 契约漂移不报错、只错位，这才是要防的。

**Q：MCP 和你们自己的 HTTP 接口有什么区别？** A：HTTP 接口是给本项目客户端用的私有约定；MCP 是标准协议，让**任意**支持它的 Agent 客户端零适配接入。一个是私有约定，一个是通用插座。